

КП 4. Віртуальний асистент з використанням LLM

Тут, у цьому тексті, під терміном **віртуальний асистент** (бот) будемо розуміти програмний застосунок, здатний спілкуватись з користувачем природною мовою (українською — обов'язково, англійською — факультативно), призначений для допомоги у виконанні певних завдань.

Передбачається, що у проєкті використовується Google Gemini API, з безкоштовним ключем (Free Tier).

Теми можна обрати з наступного списку:

(або запропонувати власну)

1. AI Code Reviewer

Бот аналізує Python-код та перевіряє його на відповідність PEP 8, виявляє потенційні вразливості та пропонує рефакторинг. Особливість: Використовуються системні інструкції для ролі "Senior Developer".

2. Multimodal Study Assistant (Telegram/Discord)

Чат-бот, якому можна надіслати фото конспекту/скриншот тексту. Бот розпізнає текст, пояснює складні терміни та генерує короткий квіз за матеріалом. Особливість Використовуються Vision-можливості Gemini Flash.

3. Smart SQL Architect

Веб-застосунок, куди користувач вводить запит природною мовою (наприклад, "покажи топ-5 покупців за минулий місяць"), а система генерує SQL-запит на основі схеми бази даних. Особливість: Робота зі структурованим виводом (JSON) та Few-shot промптингом.

4. Automatic README Generator

Інструмент, який сканує файли проєкту, розуміє структуру і автоматично створює професійний README.md з описом функцій, інструкціями з інсталяції та деревом файлів. Особливість Аналіз контексту великого обсягу коду.

5. Personalized News Digest (RAG-lite)

Програма збирає заголовки новин за обраною темою (через RSS/API) та за допомогою Gemini створює короткий аналітичний звіт: що головне, які тренди та що почитати детальніше. Особливість: Фільтрація галюцинацій та стиснення інформації.

6. AI Interviewer для технічних спеціалістів

Симулятор співбесіди, де модель виступає в ролі інтерв'юера. Вона ставить питання, оцінює відповіді та в кінці дає розгорнутий фідбек (scorecard). Особливість Використовується ChatSession для збереження контексту діалогу.

7. Context-Aware Translator для програмістів

Перекладач технічної документації або коментарів у коді, який зберігає термінологію та не перекладає ідентифікатори та слова синтаксичних конструкцій. Особливість Налаштування Stop Sequences для збереження форматування.

8. AI Test Case Generator

Користувач вводить опис функції або її код, а система генерує набір модульних тестів (Unit Tests) на PyTest, враховуючи граничні випадки (edge cases). Особливість Поєднання генерації коду та логічного аналізу вимог.

9. Semantic Search over Local Docs

Локальний пошуковик у PDF-документах. Система створює ембедінги (Embeddings) для абзаців тексту, а Gemini відповідає на питання, базуючись на знайдених фрагментах. Особливість Робота з векторними представленнями через google-gemai.

10. AI Weather & Travel Planner

Бот планує маршрут та шукає пам'яток - природних та/або інших пам'яток. Особливість: отримання реальної погоди (OpenWeather API) та пошук в Інтернеті інформації про пам'ятки.

Технічні вимоги:

1. Обов'язкове використання

- a. Streamlit (консольний варіант - факультативно);
- b. віртуальне оточення (бажано uv);

2. Рекомендована структура проєкту:

ai-project-name/

- |— .python-version # Створюється uv
- |— .venv/ # Локальне віртуальне середовище (керується uv)
- |— pyproject.toml# Головний файл конфігурації проєкту та залежностей
- |— uv.lock # Зафіксовані точні версії бібліотек (аналог package-lock.json)
- |— .gitignore # Обов'язково ігнорувати .venv
- |— src/ # Сирцевий код (рекомендований підхід)
 - | — reviewer/
 - | — __init__.py
 - | — main.py# Точка входу
 - | — core.py # Логіка взаємодії з Gemini API
 - | — schemas.py# Pydantic моделі для Structured Output
- |— tests/# Тести
 - | — test_core.py

3. Вимоги до реалізації

- a. Обробка помилок (Exception Handling): Необхідно використовувати блоки try...except для викликів до API. Потрібно окремо обробляти:
 - i. ResourceExhausted (перевищення лімітів Free Tier).
 - ii. InvalidArgument (неправильний ключ або формат).
- b. Типізація: Використання Pydantic для валідації даних та опису структури відповіді Gemini (останнє допоможе автоматично перетворити відповідь моделі на об'єкт Python).
- c. Застосування system_instruction: Роль моделі треба виносити в окремий параметр при створенні моделі.
- d. Використання (обґрунтовані значення) параметрів моделі, таких як temperature, top_p тощо.
- e. Підготовка документації (README) та презентації
- f. Вітається використання Function Calling.
- g. Вітається використання локального кешування запитів.

Шкала оцінювання

(у відсотках від максимальної кількості балів):

1. Технічна реалізація (40 %)

- a. Функціональність (20 %): Чи виконує програма заявлені функції, чи працює без помилок?
- b. Обробка помилок (10 %): Як система реагує на відсутність інтернету, невалідний API-ключ, невалідні дані тощо? Чи є try...except блоки?
- c. Архітектура (10 %): Використання uv, правильна структура папок (src/, core/) тощо.

2. Робота з Google GenAI (30 %)

- a. Якість промптів (10 %): Наскільки детально прописані System Instructions? Чи використовується рольова модель?
- b. Structured Output (10 %): Чи повертає модель валідний JSON? Чи використано response_schema (Pydantic) для стабільності?
- c. Параметри генерації (10 %): Обґрунтований вибір моделі (Flash), налаштування temperature, top_p та обробка лімітів Free Tier.

3. Якість коду та інженерна культура (20 %)

- a. Чистота коду (10 %): Дотримання PEP 8, зрозумілі назви змінних та функцій, відсутність дублювання (DRY).
- b. Документація (10 %): Наявність якісного README.md з інструкцією по встановленню, запуску через uv run та описом можливостей.

4. Презентація та захист (10 %)

- a. Демонстрація (5 %): Вміння всебічно висвітлити роботу проекту (функціональність та нетипові/критичні ситуації).
- b. Відповіді на питання (5 %): Розуміння роботи Google Ai for Developers.

Корисні посилання (див. також у презентаціях)

- 1. <https://googleapis.github.io/python-genai/genai.html>
- 2. <https://github.com/google-gemini/cookbook/tree/main/examples>
- 3. <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/function-calling#use-cases>